

1. How does Django support microservices development?

Django supports microservices through its modular design. Since each Django project is self-contained with its own settings, database connection, and URL routing, it is easy to split a large system into smaller, independent "services." Using JsonResponse allows these services to communicate using standard RESTful patterns, making it ideal for distributed architectures.

2. What advantages do containers provide for Django deployment?

Containers solve the "it works on my machine" problem. They package the specific version of Python, Django, and all system dependencies (like database drivers) into a single image. This ensures that the application runs identically in development, testing, and production environments, while also making it easy to scale by simply spinning up more container instances.

3. How would you enable service-to-service communication?

In a microservices setup, services can communicate using Synchronous methods (like one service making an HTTP/REST request to another's API using the requests library) or Asynchronous methods (using a message broker like RabbitMQ or Kafka to send events between services).

4. How can this setup be scaled using Kubernetes?

Kubernetes can scale this setup by managing "Pods" for each service. If traffic increases on the product-service, Kubernetes can automatically spin up multiple replicas of that specific container and use a Load Balancer to distribute traffic among them, ensuring high availability without affecting the user-service.